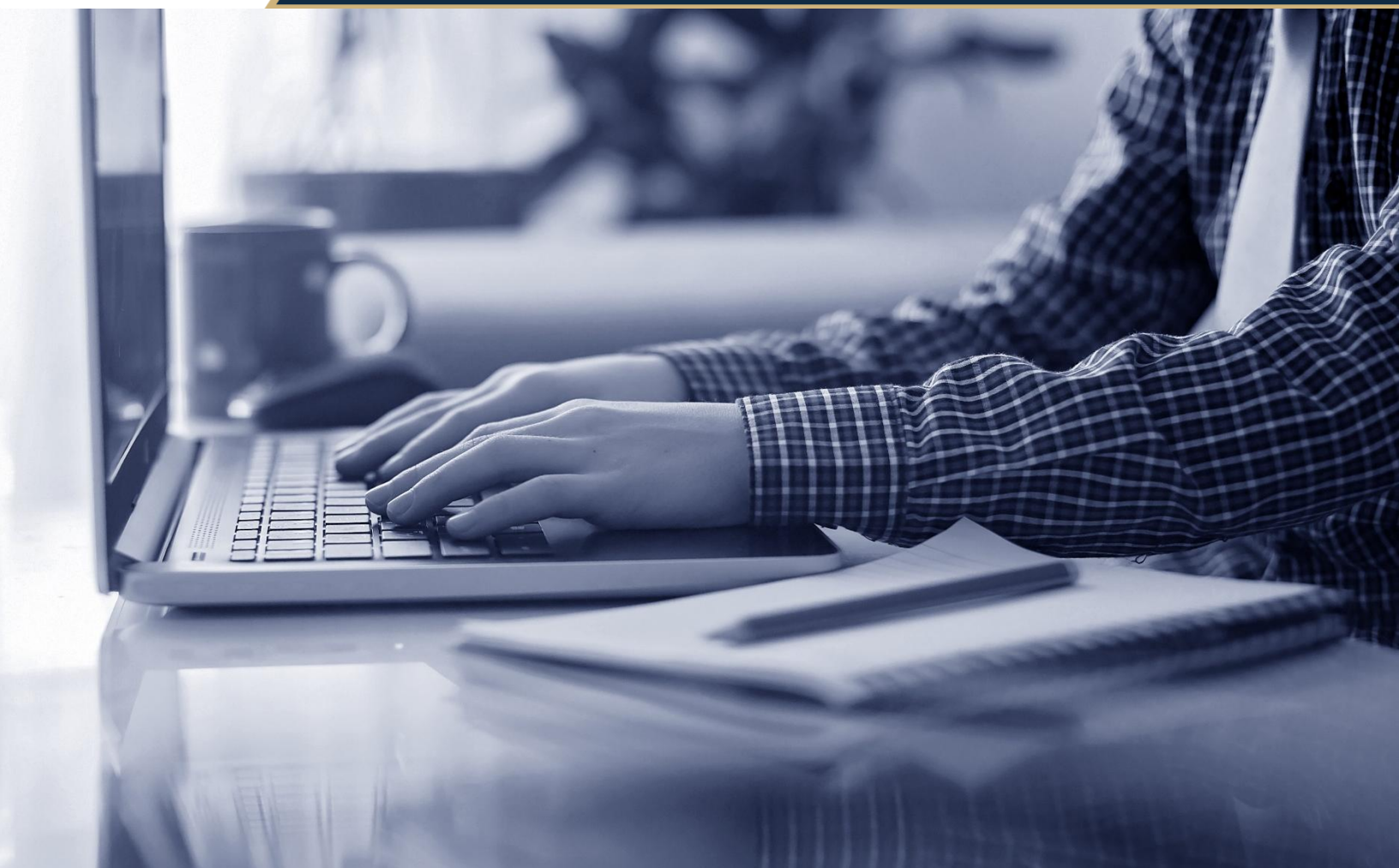




PROGRAMACIÓN PARA LA CIENCIA DE DATOS

Código: CD106ICDA

Apunte N° 3

Depuración y optimización de algoritmos para análisis de datos

Introducción

El presente documento tiene como finalidad introducir a las y los estudiantes en la depuración y optimización de algoritmos para análisis de datos, etapa tan crítica como el diseño inicial de un script o la construcción de un pipeline. En esta unidad aprenderemos a identificar, aislar y corregir errores (bugs) que afectan la exactitud de los resultados, a manejar excepciones para evitar interrupciones inesperadas, y a verificar sistemáticamente que nuestras funciones producen lo que deben producir. Del mismo modo, abordaremos técnicas para mejorar la eficiencia computacional, desde la elección de estructuras de datos adecuadas hasta la vectorización con NumPy/pandas y el uso de herramientas de temporización, con el objetivo de reducir tiempos de ejecución y uso de memoria sin sacrificar precisión.

En términos prácticos, “depurar” significa entender a profundidad el flujo y el estado del programa: dónde y por qué falla, qué suposiciones no se cumplen y cómo corregirlas. “Optimizar”, por su parte, implica reformular operaciones costosas (bucles anidados, búsquedas lineales repetidas, transformaciones redundantes) para que el mismo algoritmo haga más en menos tiempo. Ambas dimensiones —correctitud y rendimiento— avanzan de la mano: primero garantizamos que “está bien”, luego que “está mejor”.

A lo largo del apunte trabajaremos con ejemplos y casos vinculados a realidades chilenas (p. ej., limpieza y consolidación de datos de transporte público, empleabilidad regional, o calidad del aire en ciudades del sur), complementados con ejemplos internacionales cuando aporten claridad conceptual. Verás cómo aplicar depuradores, print estratégicos, bloques try/except, aserciones y pruebas unitarias para validar funciones, y cómo medir y comparar versiones de un mismo algoritmo con herramientas como timeit para cuantificar mejoras.

El resultado de aprendizaje que guía esta unidad es: “Depura los algoritmos para ajustar y mejorar la precisión y la eficiencia del análisis de datos”. Para evidenciarlo, te enfocarás en: identificar errores y limitaciones y corregirlos con técnicas de debugging, optimizar estructuras y operaciones para ganar eficiencia, ajustar parámetros (umbrales, criterios de filtrado, tamaños de lote) evaluando el impacto en la calidad de los resultados y validar lo obtenido mediante pruebas en diferentes conjuntos de datos, asegurando consistencia y robustez.

Éxito en el estudio de este apunte, el cual es clave para profundizar aún más en el uso de las tecnologías.

Seguimiento y control del proyecto

1. Depuración de código (Debugging)

La depuración de código es el proceso de encontrar y corregir errores (*bugs*) en un programa. Es una práctica indispensable en cualquier lenguaje, ya que mejorar las habilidades de depuración incrementa la calidad y confiabilidad del programa y sienta las bases para un desarrollo eficiente. El objetivo final de la depuración es lograr que el programa funcione de manera precisa y eficiente, eliminando fallos ocultos que podrían sesgar resultados o ralentizar la ejecución. En el contexto de análisis de datos, depurar es crucial para asegurar que nuestros algoritmos procesen correctamente la información.

Ejemplo práctico (Transporte público de Santiago): Imaginemos un script en Python que calcula la cantidad promedio de pasajeros por bus en Santiago, usando datos abiertos de validaciones de la tarjeta Bip! Si el resultado parece improbable (por ejemplo, un promedio muy bajo), la depuración nos permite identificar el error. Podríamos insertar instrucciones `print()` en puntos clave del cálculo para inspeccionar valores intermedios (método básico de depuración). Al revisar la salida, descubrimos que estábamos dividiendo por un número incorrecto de recorridos debido a un índice mal calculado. Corregir este bug mejora inmediatamente la precisión de nuestro análisis (las cifras ahora reflejan correctamente la realidad del transporte).

Otra técnica es usar un depurador interactivo como `pdb` en Python, colocando `pdb.set_trace()` justo antes de la sección problemática del código. Esto nos permite ejecutar el código paso a paso, inspeccionar variables y entender el flujo lógico. En nuestro ejemplo, al depurar interactivamente notamos que ciertos registros de pasajeros estaban vacíos o mal formateados, causando que el algoritmo saltara iteraciones. Con la depuración, ajustamos el manejo de esos casos especiales. Este proceso no solo corrige errores, sino que asegura que el algoritmo opere con los datos de manera confiable. En suma, la depuración contribuye directamente a mejorar la exactitud de los resultados (al eliminar cálculos erróneos) y a la eficiencia del desarrollo, al facilitar encontrar problemas temprano en el ciclo de vida del código.

2. Manejo de errores y excepciones en Python

En Python (y en general), los errores en tiempo de ejecución se manejan mediante excepciones. Un buen manejo de errores mediante bloques `try/except/finally` permite que el programa responda adecuadamente a situaciones inesperadas en lugar de abortar súbitamente. En cualquier proyecto de datos es inevitable toparse con errores (archivos faltantes, valores fuera de rango, divisiones por cero, etc.), pero si se gestionan correctamente las excepciones es posible evitar la detención inesperada del programa y mostrar mensajes de error claros al usuario. Python proporciona un mecanismo poderoso para esto: al usar manejo de excepciones, el programa gana en fiabilidad y continúa funcionando de forma controlada incluso si ocurre un problema. En otras palabras, el flujo de análisis de datos no se interrumpe ante datos faltantes o erróneos, mejorando la robustez de nuestras soluciones.

Ejemplo práctico (Datos abiertos en Chile): Supongamos que estamos analizando datos de calidad del aire obtenidos del sistema SINCA (Sistema de Información Nacional de Calidad del Aire). Es común que intentemos leer un archivo CSV diario que no existe (quizás porque una estación no reportó datos cierto día).

Sin manejo de excepciones, usar `pandas.read_csv` lanzaría una excepción y detendría completamente nuestro programa. En cambio, podemos encapsular esta operación en un bloque `try` y capturar la excepción `FileNotFoundError` con un `except`, de modo que el código registre una advertencia y continúe con el siguiente archivo. Por ejemplo:

```
import pandas as pd
```

```
try:
    df = pd.read_csv("sinca_2025-10-12.csv")
except FileNotFoundError:
    print("Aviso: datos del 12/10/2025 no disponibles, se omite ese día.")
    df = pd.DataFrame() # DataFrame vacío en reemplazo
```

De esta forma, el programa no colapsa; maneja el error de forma elegante, permitiendo que el análisis siga con los datos disponibles. Esto mejora la fiabilidad del proceso de análisis y la experiencia del usuario, quien recibe un mensaje claro en vez de un `traceback` críptico. Otro ejemplo común: si intentamos calcular un indicador y ocurre una división por cero (por ejemplo, población/área para densidad en una comuna con `área=0`), un bloque `try/except` específico puede atrapar el `ZeroDivisionError` y asignar un valor nulo o una etiqueta de "dato no aplicable". Así evitamos comportamientos inesperados o resultados inválidos en nuestros análisis. En resumen, un buen manejo de excepciones asegura que los algoritmos de datos sean más robustos, lo que contribuye a la validez de los resultados (no se detienen a medio camino) y a la confiabilidad de la herramienta analítica incluso ante datos imperfectos.

3. Pruebas y verificación de algoritmos

Implementar pruebas para verificar algoritmos es una práctica fundamental para garantizar que el código hace lo que se espera. En Python podemos usar frameworks como `unittest` o `pytest` para crear *tests* automatizados que ejercitan partes específicas del código con entradas controladas. Las pruebas unitarias permiten evaluar cada parte del código individualmente y detectar errores en etapas tempranas, siendo esenciales para asegurar la calidad del código. A medida que los programas de análisis crecen en complejidad, resulta difícil comprobar manualmente todos los casos; las pruebas sistemáticas ayudan a evitar errores inesperados causados por pequeños cambios y a mantener la estabilidad global del programa. En un contexto de ciencia de datos, esto también nos da confianza en la precisión de las funciones de procesamiento o cálculo antes de aplicarlas a grandes volúmenes de datos.

Ejemplo práctico (Verificación de cálculo estadístico): Supongamos que implementamos una función `calcular_tasa_desempleo(datos)` que, dado un `DataFrame` con la población activa y desocupada en Chile por trimestre, calcula la tasa de desempleo. Podemos escribir pruebas unitarias para esta función con escenarios simples conocidos. Por ejemplo, una prueba con datos ficticios donde 100 personas de la fuerza laboral y 7 desocupadas debería devolver 7% de desempleo. En `unittest` sería algo así:

```
import unittest
# Función bajo prueba
def calcular_tasa_desempleo(poblacion_activa, desocupados):
    return (desocupados / poblacion_activa) * 100
class TestCalculosEconomicos(unittest.TestCase):
    def test_tasa_desempleo_basica(self):
        tasa = calcular_tasa_desempleo(100, 7)
        self.assertEqual(round(tasa, 2), 7.00)
```

Al ejecutar la prueba, verificamos que la función entrega 7.00% como se espera. Podríamos añadir más casos: todos empleados (0%), mitad desempleados (50%), etc. Estas pruebas exhaustivas validan la precisión del algoritmo en distintos escenarios, incluyendo bordes (¿qué pasa si `poblacion_activa=0`? Debería manejarse). Otro caso, a un nivel más avanzado, sería probar funciones de limpieza de datos: por ejemplo, que una función `limpiar_rut(rut)` efectivamente elimine puntos y guiones de un RUT chileno correctamente. Con datos de ejemplo conocidos (RUT "12.345.678-9" debería quedar "123456789"), escribimos test para confirmar el comportamiento.

Al integrar estas pruebas en nuestro flujo de trabajo, cada vez que modifiquemos el código podemos correr la batería de test. Así nos aseguramos de no introducir regresiones: si un cambio rompe algo que antes funcionaba, una prueba fallará alertándonos inmediatamente. Esto aporta un alto grado de validación a nuestros algoritmos, porque sabemos que han sido verificados con casos de uso relevantes. En suma, las pruebas y verificación contribuyen tanto a la precisión (detectan desviaciones o cálculos erróneos) como a la confiabilidad (garantizan que el código sigue cumpliendo los requisitos tras optimizaciones o refactorizaciones).

4. Técnicas de optimización de código

En análisis de datos es habitual trabajar con grandes volúmenes de información (millones de registros de sensores, transacciones, etc.), por lo que la eficiencia del código es clave. La optimización de código busca mejorar el rendimiento (tiempo de ejecución y uso de recursos) de nuestros programas sin alterar el resultado final. Un software optimizado puede procesar más datos en menos tiempo, lo que maximiza la productividad y permite análisis más profundos. Antes de optimizar, es recomendable realizar un *análisis de rendimiento* (profiling) para identificar los cuellos de botella, evitando la "optimización prematura" de partes no críticas. Identificados los puntos lentos, podemos aplicar diversas técnicas de optimización en Python, tales como:

- Usar funciones y bibliotecas optimizadas: Aprovechar las funciones integradas de Python y librerías como NumPy/Pandas en lugar de reimplementar lógica en bucles de Python puro. Estas funciones nativas suelen estar escritas en C y han sido probadas por la comunidad, evitando "reinventar la rueda" y reduciendo código innecesario. Por ejemplo, para sumar una columna de millones de filas, `df["col"].sum()` de pandas utilizará código optimizado en C, mucho más rápido que iterar fila por fila con un `for`.
- Evitar bucles y operaciones innecesarias: Reemplazar, cuando sea posible, los bucles explícitos por operaciones vectorizadas o comprensiones de listas. Los bucles en Python conllevan sobrecarga interpretativa; en cambio, una comprensión de lista ejecuta la operación en código interno optimizado. Por ejemplo:
 - Si queremos convertir una lista de temperaturas en Celsius a Fahrenheit, es más eficiente `fahrenheit = [c*9/5 + 32 for c in temps_c]` que un `loop` acumulativo. Esto reduce significativamente el tiempo de ejecución y hace el código más conciso. Asimismo, funciones *built-in* como `sum(lista)` o expresiones generadoras suelen ser más eficientes que código manual equivalente.
- Operaciones en lote (vectorización): En manejo de datos numéricos, apoyarse en librerías como NumPy (arreglos) o pandas (DataFrames) permite realizar cálculos masivos en lote aprovechando implementaciones optimizadas en bajo nivel. Por ejemplo, calcular la media por columna de una matriz grande es casi instantáneo con `numpy.mean(array, axis=0)` en lugar de múltiples bucles. En

un caso de análisis de redes de transporte, usar pandas para agrupar y resumir datos (con métodos como `.groupby()` y `.agg()`) es más eficiente que agrupar manualmente con estructuras Python.

- **Memorización y caché:** Si un cálculo costoso se repite con frecuencia, podemos almacenar sus resultados para reutilizarlos. La memorización consiste en guardar en caché la salida de funciones para entradas dadas. En aplicaciones de ciencia de datos, un ejemplo podría ser almacenar la distancia entre dos direcciones ya calculada, en lugar de recalcularla cada vez que aparece la misma pareja en un algoritmo de logística. Python ofrece decoradores como `functools.lru_cache` para facilitar esta técnica.
- **Paralelismo y concurrencia:** Para tareas muy pesadas, dividir el trabajo entre varios núcleos de CPU puede reducir tiempos. Python permite usar la biblioteca `multiprocessing` o librerías como `Joblib` o `Dask` para ejecutar operaciones en paralelo. Por ejemplo, al preprocesar millones de fotos de satélite, podríamos distribuir las imágenes entre procesos concurrentes. Esto aprovecha mejor el hardware disponible, aunque debemos tener en cuenta la sobrecarga de comunicación entre procesos y el *Global Interpreter Lock* (GIL) de Python (que en cálculo puro Python limita la `threading`). El paralelismo es especialmente útil cuando se integran llamadas a código externo (como funciones de NumPy, que liberan el GIL o código compilado).
- **Perfilado y monitoreo de rendimiento:** Una parte integral de la optimización es medir. Herramientas de perfilado (profilers) como `cProfile` o `line_profiler` nos indican qué funciones o líneas consumen más tiempo o memoria. Por ejemplo, al analizar el proceso de limpieza de datos de empleo del INE, podríamos descubrir que convertir tipos de datos con `astype` ocupa el mayor tiempo. Con esa información, aplicamos técnicas (como convertir tipos en bloque en lugar de celda a celda) y luego volvemos a medir para cuantificar la mejora. Este ciclo iterativo nos lleva a un código cada vez más eficiente.

Imagen 1: Resumen de estrategias de optimización de código Python desde lo más simple a lo más complejo.



Fuente: Imagen creada con IA

Ejemplo práctico (Optimización en datos de Santiago): Consideremos un análisis de datos de movilidad donde tenemos todos los registros de validaciones de transporte público de un mes (millones de filas con hora, estación, bus, tarjeta, etc.). Una versión inicial del código usa bucles anidados en Python para contar abordajes por estación y por hora punta. Este enfoque tarda, digamos, 30 minutos en correr. Aplicando optimizaciones: primero identificamos que el cuello de botella son los bucles; reescribimos el cálculo usando pandas: `df.groupby(['estacion','hora']).size()`. Este solo cambio reduce el tiempo a 1 minuto gracias a la vectorización. Además, usamos un diccionario para mapear los códigos de estación a nombres durante el conteo en lugar de realizar búsquedas repetitivas en una lista. Tras optimizar, evaluamos que los resultados (conteos por estación/hora) sean los mismos que antes – validando que la optimización no introdujo errores – y comprobamos que la eficiencia mejoró drásticamente (30x más rápido). Como se observa, estas técnicas de optimización permiten manejar conjuntos de datos más grandes en menos tiempo, incrementando la eficiencia sin sacrificar la precisión del análisis.

5. Optimización de estructuras de datos

Relacionado con el punto anterior, la elección adecuada de estructuras de datos puede marcar una gran diferencia en el rendimiento de un algoritmo. En Python disponemos de listas, tuplas, conjuntos (*sets*), diccionarios (*dict*), colas (*deque*), entre otras, cada una con propiedades de eficiencia distintas. Usar la estructura correcta para la tarea correcta mejora significativamente la eficiencia en tiempo y memoria. Por ejemplo, buscar un elemento en una lista es una operación $O(n)$ (lineal en el tamaño), mientras que buscar en un conjunto o en las claves de un diccionario es $O(1)$ amortizado (básicamente constante), gracias a que están implementados con tablas hash.

Ejemplo práctico (Set vs Lista en validación de datos): Supongamos que trabajamos con datos de usuarios del sistema de salud digital en Chile (por ejemplo, IDs de pacientes que han iniciado sesión en el portal). Tenemos una lista de millones de registros de inicio de sesión, y necesitamos filtrar cuáles son de pacientes de la Región Metropolitana. Si tenemos los RUT o IDs de todos los pacientes de la RM en una lista y para cada inicio de sesión comprobamos si el ID está "en la lista", el rendimiento será muy bajo, porque por cada búsqueda la lista se recorre parcialmente. En cambio, si almacenamos los IDs de la RM en un set, cada consulta de pertenencia será mucho más rápida. Efectivamente, usar un conjunto en lugar de una lista mejora el rendimiento en operaciones de búsqueda (*membership*) de forma drástica. En una prueba de concepto, filtrar 1 millón de registros contra 10 mil IDs conocidos podría tardar segundos con un set, versus minutos con una lista.

Otro ejemplo: en análisis de transporte, si necesitamos asociar rápidamente cada código de estación con sus coordenadas, un diccionario `{codigo_estacion: (lat, lon)}` nos permite acceder por clave instantáneamente, a diferencia de buscar en una lista de tuplas o en un DataFrame sin índice (que sería lineal). Los diccionarios y sets en Python están optimizados para búsquedas rápidas y son ideales para estructuras de tipo clave-valor. Por su parte, las listas son eficientes para recorridos secuenciales y para agregar elementos al final, pero no tanto para búsqueda; mientras que las tuplas, al ser inmutables, pueden usarse como claves en diccionarios o sets.

También es importante considerar la estructura de datos de las librerías científicas. Por ejemplo, si manejamos datos numéricos, un array de NumPy es más eficiente que una lista de Python porque almacena elementos contiguos en memoria y permite vectorización. Un *DataFrame* de pandas es eficiente para operaciones columnares y ofrece índices para acceso rápido a subconjuntos de datos. En resumen, elegir estructuras de datos adecuadas según la necesidad puede reducir el uso de memoria y el tiempo de cómputo significativamente. Esta optimización contribuye a la eficiencia global del algoritmo, permitiendo escalar a conjuntos de datos mayores sin degradar tanto el rendimiento. Además, usar la

estructura correcta también puede mejorar la legibilidad del código (ej.: un conjunto comunica claramente que nos importan elementos únicos, un diccionario comunica una relación clave-valor), lo que enlaza con las buenas prácticas de código.

(*Contexto internacional:* En aplicaciones de datos a nivel global, los mismos principios aplican. Por ejemplo, en un análisis de redes sociales, utilizar un diccionario para mapear usuario->lista de amigos permite consultas directas sobre la lista de amigos de un usuario dado, en comparación a buscar en una lista global de conexiones cada vez. Para una plataforma con millones de usuarios, esta decisión de estructura es la diferencia entre una consulta que tarda microsegundos contra segundos, escalando a mejoras enormes en eficiencia.)

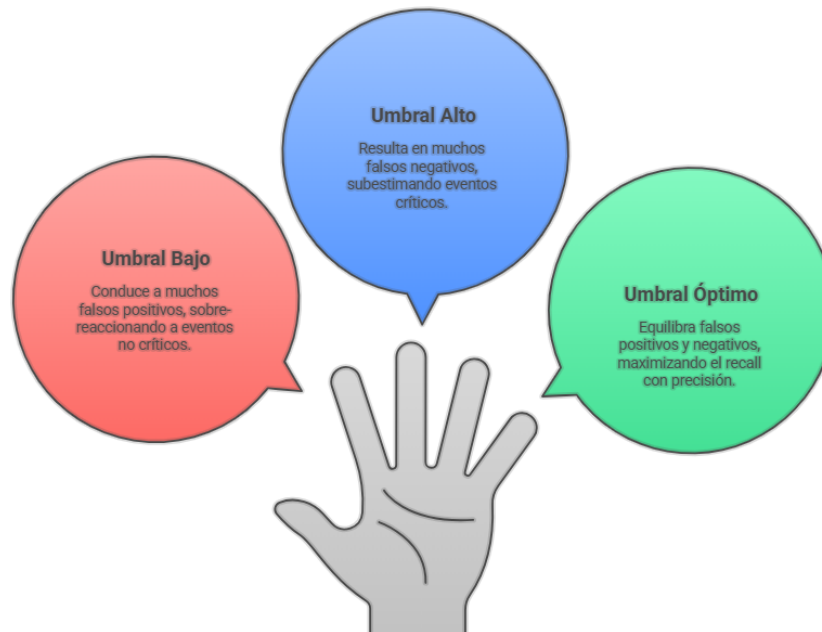
6. Ajuste y calibración de algoritmos

No basta con tener un algoritmo que funcione; en ciencia de datos a menudo necesitamos ajustar o calibrar el algoritmo para obtener los mejores resultados. Esto se refiere típicamente a afinar los parámetros o hiperparámetros de un algoritmo para mejorar su desempeño. Un hiperparámetro es una variable de configuración externa al modelo (por ejemplo, la profundidad máxima de un árbol de decisión, el número de *clusters* en k-means, el coeficiente de regularización en una regresión, o incluso un umbral manual en una regla de negocio). Encontrar la configuración óptima suele requerir experimentación y validación.

En el contexto de *machine learning*, el proceso de buscar el mejor conjunto de hiperparámetros se conoce como ajuste de hiperparámetros (*hyperparameter tuning*). Este proceso es fundamental porque los hiperparámetros controlan directamente la estructura y el rendimiento de los modelos, y ajustarlos correctamente permite lograr resultados óptimos. Por ejemplo, si se entrena un modelo de bosques aleatorios para predecir la contaminación del aire en Santiago, parámetros como el número de árboles (*n_estimators*) o la profundidad máxima de cada árbol (*max_depth*) influirán en la precisión de las predicciones. Un valor muy bajo de *max_depth* puede subestimar patrones (modelo poco preciso), mientras uno demasiado alto puede sobreajustar el modelo a los datos de entrenamiento y hacerlo más lento. Elegir valores adecuados y balanceados de hiperparámetros produce modelos más precisos y con buen rendimiento.

Ejemplo práctico (Calibración de modelo con datos chilenos): Imaginemos un algoritmo de clasificación que predice si una estación de monitoreo de aire estará en estado de alerta ambiental al día siguiente, basado en datos históricos (temperatura, emisiones, etc.). Este modelo tiene un hiperparámetro de umbral de probabilidad para decidir "Alerta" vs "No Alerta". Si fijamos el umbral muy bajo, prácticamente todas las predicciones serán "Alerta" (muchos falsos positivos); si es muy alto, el modelo solo alertará en casos extremos (muchos falsos negativos). Debemos calibrar ese umbral analizando resultados en un conjunto de validación: probamos, por ejemplo, umbral = 0.5, 0.6, 0.7, etc., y escogemos aquel que maximiza la métrica deseada (digamos, *recall* con respecto a alertas reales, manteniendo un cierto nivel de precisión). Esta calibración mejora la precisión de los resultados del algoritmo en el contexto de políticas públicas, evitando sobre-reacción o subestimación de eventos de alerta.

Imagen 2: resumen sobre los umbrales de probabilidad óptimo para el algoritmo de alerta ambiental



Fuente: Imagen creada con IA

Otro ejemplo: en algoritmos de optimización de rutas de transporte público, podríamos tener un parámetro de "penalización de trasbordos". Calibrarlo (aumentando o disminuyendo el peso de cada trasbordo en el coste de la ruta) afectará las rutas óptimas sugeridas. Ajustamos este parámetro comparando las rutas resultantes con datos reales de viajes de usuarios: encontramos el valor que produce rutas similares a las realmente utilizadas (lo cual valida que nuestro modelo refleja bien el comportamiento real).

Existen enfoques sistemáticos para el ajuste de algoritmos, como búsqueda en grilla (grid search), búsqueda aleatoria, o métodos más inteligentes como optimización bayesiana de hiperparámetros. En cualquier caso, el proceso implica entrenar o ejecutar repetidamente el algoritmo con diferentes configuraciones, medir su desempeño (ej.: error de predicción, tiempo de cómputo, etc.) y escoger la mejor. Esto puede ser computacionalmente costoso, pero los beneficios suelen ser grandes: un algoritmo bien ajustado puede aumentar significativamente la precisión de un modelo predictivo o la eficiencia de un algoritmo de búsqueda. Cabe destacar que este ajuste debe hacerse con cuidado, usando validación cruzada o conjuntos de prueba separados, para evitar optimizar el algoritmo solo para un subconjunto de datos y perder generalización.

(*Contexto internacional:* Un ejemplo famoso de ajuste de algoritmos ocurre en competencias de *data science* como Kaggle, donde los participantes afinan hiperparámetros de modelos de aprendizaje automático para expresar la máxima precisión posible. Asimismo, en la industria, servicios en la nube como AWS SageMaker ofrecen herramientas automáticas de *hyperparameter tuning*, reconociendo la importancia de calibrar modelos para alcanzar altos estándares de rendimiento.)

En resumen, ajustar y calibrar algoritmos es un paso vital para mejorar la precisión de los análisis de datos y asegurar que el algoritmo trabaja en condiciones óptimas. Es la diferencia entre un modelo que simplemente funciona y uno que está validado y optimizado para el contexto específico (ya sea local, como datos chilenos, u otros escenarios).

7. Evaluación de resultados y eficiencia

Una vez desarrollado un algoritmo o modelo, es fundamental evaluar sus resultados y su eficiencia. La evaluación de resultados responde a preguntas como: *¿Qué tan preciso es el algoritmo? ¿Sus salidas son correctas y útiles?* Mientras que la evaluación de eficiencia considera: *¿Qué tan rápido o escalable es el algoritmo? ¿Consume recursos razonables?* En la práctica, estos dos aspectos van de la mano para validar plenamente una solución de ciencia de datos.

Evaluación de resultados (precisión y validación): Implica verificar que el algoritmo produce resultados confiables y exactos. Esto generalmente se logra mediante pruebas y comparaciones contra referencias. La verificación de algoritmos es un proceso sistemático que garantiza su precisión y fiabilidad, involucrando pruebas exhaustivas, validación en diferentes escenarios y detección de sesgos o errores. Por ejemplo, si hemos desarrollado un algoritmo de clasificación de imágenes satelitales para identificar zonas de vegetación en Chile, evaluaremos su precisión comparando sus etiquetas con imágenes ya categorizadas manualmente (ground truth). Usaremos métricas cuantitativas (exactitud, F1, MAE, etc. según el caso) para resumir qué tan cerca estamos del resultado deseado. También realizaremos validación cruzada o pruebas A/B cuando aplique, asegurando que el desempeño se mantiene en diferentes subconjuntos de datos. Parte de esta evaluación es probar el algoritmo en diversos escenarios: por ejemplo, en el caso de un algoritmo de predicción meteorológica, verificar que funcione tanto en verano como en invierno, en climas del norte grande y del sur, etc., para garantizar su robustez.

Un checklist de evaluación incluiría: precisión de los resultados (*¿produce la salida correcta para entradas conocidas?*), relevancia (*¿los resultados responden a la pregunta de negocio o investigación que tenemos?*), y ausencia de errores obvios. Por ejemplo, validar un algoritmo de ordenamiento verificando que efectivamente ordena listas de prueba correctamente de menor a mayor, incluyendo casos extremos como listas ya ordenadas, lista vacía, lista con valores repetidos, etc. Si alguno de estos casos falla, sabemos que hay un error que corregir. En términos de validación de modelos predictivos, se suele reservar un conjunto de datos de prueba no usados en el entrenamiento para confirmar que el modelo generaliza bien (no solo memorizó patrones particulares del conjunto de entrenamiento). Todo esto garantiza la fiabilidad e integridad de la información generada por el algoritmo antes de su uso en producción.

Evaluación de eficiencia: Un algoritmo preciso pero que toma días en correr puede ser inviable en la práctica. Por ello, evaluamos también el desempeño computacional. Esto puede implicar medir el tiempo de ejecución con diferentes tamaños de datos (para evaluar la escalabilidad) y monitorear el uso de recursos (CPU, memoria). En proyectos de datos reales, solemos hacernos preguntas como: *¿Cuánto tarda nuestro script en procesar un día de datos de sensores? ¿y un mes?* Si el tiempo crece exponencialmente con el tamaño, tal vez el algoritmo no escale bien. Herramientas de perfilado (como mencionamos en la optimización) ayudan a obtener estas métricas. También es útil comparar la eficiencia con alternativas: por ejemplo, si implementamos un algoritmo de ordenación personalizado, podemos compararlo con el algoritmo nativo (sorted) en Python para ver si vale la pena nuestra solución. Incluir la eficiencia en la validación es importante; de hecho, profesionales suelen incluir factores como escalabilidad y eficiencia en sus verificaciones de algoritmos.

Ejemplo práctico (Evaluando un algoritmo de datos de aire): Supongamos que desarrollamos un algoritmo que predice el nivel de material particulado (PM2.5) del aire para el día siguiente en distintas comunas, usando datos históricos. Para evaluar sus resultados, calculamos el error promedio (por ejemplo, RMSE) comparando las predicciones con los valores reales observados. Vemos que el RMSE es de $5 \mu\text{g}/\text{m}^3$, lo cual podría ser aceptable o no dependiendo de estándares de calidad. Además, verificamos casos: en días de invierno con alta contaminación, ¿el modelo logra anticipar correctamente

los peaks? Esto es importante para la precisión y utilidad del modelo (validación cualitativa adicional). Luego evaluamos eficiencia: ¿podemos correr el modelo para las 52 estaciones del país en menos de, digamos, 1 minuto? Si usar una cierta técnica de modelado toma 10 minutos por estación, quizás debamos buscar optimizaciones (vectorizar la predicción por comuna, o usar cómputo paralelo para varias comunas a la vez).

Otro ejemplo: Después de optimizar el código de conteo de pasajeros por estación en Santiago (en el punto 4), queremos asegurarnos de no haber sacrificado exactitud. Comparamos los resultados del código optimizado con los del código original (aunque lento) en un subconjunto de datos. Si coinciden, validamos que la optimización preservó la precisión. Luego medimos el tiempo de ejecución en todo el dataset: confirmamos la mejora de eficiencia (de 30 min a 1 min en el ejemplo). Esta evaluación de resultados y eficiencia nos da confianza para usar el algoritmo en producción, sabiendo que entrega resultados correctos en un tiempo razonable y escalable. En términos generales, una buena práctica es elaborar una lista de verificación que considere: exactitud, escalabilidad (pruebas con distintos tamaños de datos) y comparativa de rendimiento con otras soluciones, para asegurarnos que el algoritmo es "de primera categoría" en todas estas dimensiones.

En conclusión, evaluar tanto la calidad de los resultados como la eficiencia computacional cierra el ciclo de desarrollo: confirma que hemos logrado un balance óptimo entre precisión (y validación de que los resultados son correctos) y eficiencia (el algoritmo es viable y óptimo en recursos). Si la evaluación revela deficiencias, iteramos mejorando el algoritmo (p. ej., volvemos a depurar, optimizar o incluso refactorizar) hasta cumplir con los estándares deseados.

8. Buenas prácticas de refactorización de código

La refactorización de código es el proceso de *reestructurar* internamente el código fuente para mejorar su legibilidad, mantenibilidad y eficiencia, sin cambiar su funcionalidad externa. Es comparable a corregir y pulir un documento escrito: el contenido (el resultado que produce el programa) permanece igual, pero la redacción (el código) se vuelve más clara y organizada. Refactorizar implica aplicar métodos como renombrar variables/métodos para que sean descriptivos, extraer trozos repetidos en funciones reutilizables, simplificar lógica compleja dividiéndola en partes más manejables, entre otros. Cada vez más desarrolladores y organizaciones optan por refactorizar código existente en lugar de reescribirlo desde cero, ya que es una forma económica y segura de optimizar un software a largo plazo, haciéndolo más legible y claro para otros programadores.

Beneficios técnicos: Al mejorar la estructura, reducimos lo que se conoce como *code smells* (código con señales de problemas futuros, como duplicación, funciones demasiado largas, dependencias enredadas). Un código refactorizado tiende a ser más fácil de depurar y de probar, pues está organizado en módulos o funciones pequeñas con responsabilidades claras. También facilita la implementación de optimizaciones: por ejemplo, si aislamos en una función una parte del cálculo, luego podemos optimizar internamente esa función sin temor a romper otras partes. Además, la refactorización continua previene la degradación hacia el "código espagueti" (código desordenado y difícil de seguir), manteniendo la fiabilidad del programa en evolución. Un código limpio es menos propenso a errores porque es más comprensible; nuevos desarrolladores en el equipo pueden entenderlo y modificarlo sin introducir bugs involuntariamente.

Ejemplo práctico (Refactorización en análisis de datos públicos): Imaginemos un script que consulta distintas APIs de datos públicos chilenos (transporte, clima, población) y luego combina la información. La primera versión del código fue creciendo orgánicamente y terminó con 300 líneas en un solo bloque, con variables como x, y, z y secciones duplicadas para cada API. Decidimos refactorizarlo:

- Creamos funciones separadas, por ejemplo, obtener_datos_transporte(), obtener_datos_clima(), etc., cada una encargada de llamar a su API y retornar un DataFrame.
- Estandarizamos nombres de variables (por ej., usamos df_transporte, df_clima en lugar de nombres genéricos) para que el código sea autodescriptivo.
- Extraemos la lógica duplicada de limpieza (quizás cada dataset necesita convertir fechas al mismo formato) en una función limpiar_fecha(df) que reutilizamos en cada conjunto de datos.
- Agregamos docstrings y comentarios donde la intención no es obvia, explicando, por ejemplo, que obtener_datos_transporte usa la API oficial de datos abiertos de RED (ex Transantiago) y pagina sobre los resultados JSON.

Tras refactorizar, el código tiene ahora una estructura modular más clara. El flujo principal se reduce a unas pocas llamadas a funciones, lo que mejora la legibilidad. Si en el futuro hay que cambiar cómo se obtiene un dataset (porque la API cambió), sabemos exactamente en qué función tocar sin arriesgar el resto. Importante: verificamos que el programa refactorizado produce exactamente la misma salida que antes – como la refactorización no debe cambiar la funcionalidad, los resultados deben mantenerse (aquí es útil contar con pruebas unitarias que confirmen que todo sigue bien después de refactorizar).

A continuación, un ejemplo de código en Python que funciona, pero con problemas de legibilidad y mantenimiento:

```
import pandas as pd

# Leer archivo de datos
df = pd.read_csv("ventas.csv")

# Filtrar solo productos de tipo A
df = df[df["tipo_producto"] == "A"]

# Convertir precios con coma a punto y a float
df["precio"] = df["precio"].str.replace(",", ".").astype(float)

# Calcular total
totales = []
for i in range(len(df)):
    totales.append(df.iloc[i]["precio"] * df.iloc[i]["cantidad"])
df["total"] = totales

# Guardar resultados
df.to_csv("ventas_filtradas.csv", index=False)
```

Este código tiene algunos problemas:

- Se sobrescribe el df sin modularizar ni documentar pasos.
- Se realiza una modificación destructiva sin respaldo.
- El cálculo del total se hace con un for y .iloc, lo cual es ineficiente.
- No hay funciones reutilizables ni separación de responsabilidades.
- No hay manejo de errores (por ejemplo, si falta la columna precio).
- El cambio de formato de texto a float puede fallar sin control.

Código refactorizado:

```
import pandas as pd

def cargar_datos(ruta_archivo):
    """Carga el archivo CSV en un DataFrame"""
    try:
        return pd.read_csv(ruta_archivo)
    except FileNotFoundError:
        print("Error: El archivo no existe.")
        return None

def filtrar_tipo(df, tipo="A"):
    """Filtra productos por tipo"""
    return df[df["tipo_producto"] == tipo].copy()

def normalizar_precios(df):
    """Convierte los precios a formato numérico (coma a punto)"""
    try:
        df["precio"] = df["precio"].str.replace(",", ".", regex=False).astype(float)
    except Exception as e:
        print(f"Error al convertir precios: {e}")
    return df

def calcular_totales(df):
    """Calcula el total por fila usando vectorización"""
    df["total"] = df["precio"] * df["cantidad"]
    return df

def guardar_datos(df, ruta_salida):
    """Guarda el DataFrame resultante en CSV"""
    df.to_csv(ruta_salida, index=False)
    print(f"Archivo guardado como {ruta_salida}")

# Programa principal
def main():
    df = cargar_datos("ventas.csv")
    if df is not None:
        df = filtrar_tipo(df, tipo="A")
        df = normalizar_precios(df)
        df = calcular_totales(df)
        guardar_datos(df, "ventas_filtradas.csv")

if __name__ == "__main__":
    main()
```


Cambios y sus beneficios

Mejora	Descripción	Beneficio
Modularidad	Cada paso se encapsuló en funciones (cargar_datos, filtrar_tipo, etc.).	Reutilizable, testeable y claro.
Vectorización	Se eliminó el for y .iloc, usando operaciones vectorizadas (df["precio"] * df["cantidad"]).	Código más rápido y conciso.
Control de errores	Manejo de excepciones al leer el archivo y convertir precios.	Evita que el programa falle inesperadamente.
Documentación	Cada función tiene docstrings que explican su propósito.	Mejora la mantenibilidad del código.
Evita sobrescribir variables	df se copia cuando se filtra.	Reduce riesgos de errores no intencionados.

Además de la legibilidad, una buena refactorización puede revelar oportunidades de optimización. Por ejemplo, al separar la obtención de datos de su procesamiento, nos damos cuenta de que podemos paralelizar las llamadas a APIs (porque son independientes). O notamos que ciertas transformaciones se repetían, y ahora podemos aplicar una sola vez a la concatenación final de datos. Esto reduce cómputo redundante, mejorando la eficiencia.

Un conjunto de buenas prácticas de refactorización incluye: eliminar código muerto (secciones que ya no se usan), no duplicar código (principio DRY: *Don't Repeat Yourself*, factorizar lo común), mantener funciones cortas y enfocadas en una tarea, usar nombres claros, y adherirse a convenciones de estilo (como PEP8 en Python) para uniformidad. Herramientas de análisis estático pueden ayudar a detectar puntos mejorables. La refactorización frecuente, sumada a un buen suite de tests, asegura que el código se mantenga limpio y confiable a medida que el proyecto de datos crece o cambia.

En términos de los objetivos de aprendizaje de esta unidad, las buenas prácticas de refactorización demuestran la capacidad de mejorar un código sin alterar su salida, incrementando la mantenibilidad y potencialmente la eficiencia. Un código bien refactorizado facilita la incorporación de nuevas funcionalidades de análisis de datos en el futuro y reduce la probabilidad de errores (aumentando la precisión de futuros desarrollos al minimizar errores humanos). Como analogía mencionada, refactorizar es como editar un libro para que el texto, sin cambiar su significado, sea más entendible para todos. Esto redundará en proyectos de ciencia de datos más sostenibles, donde colaborar, escalar y validar se vuelven tareas más sencillas gracias a un código fuente de calidad.

Preguntas de activación del aprendizaje:



COMPRUEBO MI APRENDIZAJE

1. ¿Por qué es importante usar instrucciones como `print()` o herramientas de depuración en un algoritmo de análisis de datos?
2. ¿Qué ventaja tiene usar estructuras como `set` o `dict` en vez de listas cuando se busca eficiencia en grandes volúmenes de datos?
3. ¿Cómo se valida que una optimización aplicada a un algoritmo no afectó negativamente sus resultados?

En el siguiente QR puedes revisar las respuestas esperadas



Información complementaria para el aprendizaje:



Información complementaria

Link 1: Depuración en Python con `pdb` (Python Debugger)
<https://realpython.com/python-debugging-pdb/>

Link 2: Manejo de errores y excepciones en Python
<https://docs.python.org/es/3/tutorial/errors.html>

Link 3: Optimización de código con vectorización en `pandas` y `NumPy`
<https://blog.sergiomarquez.dev/post/optimizacion-vectorizacion-numpy-calculos-avanzados-ia-20250515>

Conclusión

En este apunte activador, cada uno de los contenidos revisados – depuración, manejo de errores, pruebas, optimización de código y estructuras, ajuste de algoritmos, evaluación de resultados y refactorización – contribuye de manera complementaria a mejorar la precisión, eficiencia y validación en proyectos de programación para ciencia de datos. Los ejemplos contextualizados en realidades chilenas demuestran cómo estas técnicas se aplican en problemas concretos (transporte, calidad del aire, servicios digitales), evidenciando su importancia práctica. Al dominar estas herramientas, el estudiante podrá desarrollar análisis de datos más confiables (libres de bugs y resultados erróneos), más eficientes (capaces de procesar datos masivos en tiempos acotados) y más verificables (con pruebas y validaciones que respaldan las conclusiones). En suma, esta unidad sienta una base técnica sólida para enfrentar desafíos de programación en ciencia de datos del mundo real, mejorando continuamente la calidad del código y de los resultados obtenidos.

Referencia Bibliografía

López, A., & Rojas, A. L. (2021). Introducción a Python para estudiantes de ciencias. Editorial de la Universidad Pedagógica y Tecnológica de Colombia (UPTC). <https://ipss.odilo.us/info/introduccion-a-python-para-estudiantes-de-ciencias-02608297>

Kelleher, J. D., & Tierney, B. (2021). Ciencia de datos (Serie de conocimientos esenciales de MIT Press). Ediciones UC. <https://ipss.odilo.us/info/ciencia-de-datos-la-serie-de-conocimientos-esenciales-de-mit-press-02574729>

Referencia del presente documento:

Instituto San Sebastián, Departamento Online (2025). *Apunte 3: Depuración y optimización de algoritmos para análisis de datos*.